

INSTITUCIÓN

Programación de Software

Enunciado del examen

POO con Python, PEP8 y flujo Git (ramas protegidas y **feat**)

1. Objetivo

Desarrollar un proyecto en Python aplicando **Programación Orientada a Objetos** (POO), siguiendo las buenas prácticas **PEP 8** con formateador **Black** obligatorio, y utilizando **Git** con un flujo basado en ramas protegidas **dev**, **qa** y **prod** (la rama **main** no se usa ni se protege) y ramas de aporte **feat/...**

2. Fechas y modalidad

- **Inicio de entrega:** Viernes 20 de febrero de 2025.
- **Fecha límite:** Viernes 27 de febrero de 2025, a las **12:00 del mediodía**. Después de esa hora no se tendrán en cuenta nuevos **push** ni **commits**; solo se evaluará el estado del repositorio hasta ese momento.
- **Nota:** El viernes 27 de febrero **no habrá clase**.
- **Entrega formal:** es **obligatorio** registrar en el archivo Excel que el docente comparte en el equipo/grupo de Teams, junto al nombre de cada estudiante: la **URL del repositorio GitHub** y el **nombre de la cuenta de GitHub**. Sin la URL y el usuario de GitHub en el Excel no se considera entregado.

3. Requisitos técnicos

3.1 Programación Orientada a Objetos (Python)

- Uso de clases con encapsulamiento (atributos privados/protegidos donde corresponda).
- Herencia y reutilización de código (clase base y subclases).
- Métodos con responsabilidad clara y tipado con type hints (**->None**, **->str**, etc.).
- Estructura modular (por ejemplo **src/entities/**, módulos reutilizables).

3.2 Estilo y buenas prácticas (PEP 8) y Black

- **Uso obligatorio del formateador black:** el código debe estar formateado con Black para garantizar el cumplimiento de PEP 8. Se recomienda ejecutar **black** . en la raíz del proyecto antes de cada commit.
- Nombres en **snake_case** para funciones y variables; **PascalCase** para clases.

- Líneas según convención Black (máximo 88 caracteres por defecto).
- Docstrings en módulos y funciones relevantes.
- Sin `try/except` innecesarios; validación explícita de entradas cuando se indique.

3.3 Git: ramas protegidas, flujo y nombre del repositorio

- **Nombre del repositorio:** debe comenzar por `backend` y luego el nombre del proyecto (ej.: `backend-banco`, `backend-sistema-inventario`).
- **Rama main:** **no** va protegida y **no** se utiliza en este flujo; el trabajo se hace solo sobre `dev`, `qa` y `prod`.
- **Ramas protegidas (las que sí se usan):**
 - `dev`: desarrollo integrado.
 - `qa`: ambiente de pruebas.
 - `prod`: producción (o reflejo de entrega final).
- **Ramas de aporte:** cada estudiante debe trabajar en **ramas** `feat/cambio-que-hizo`, es decir, el nombre de la rama describe el cambio realizado. Por ejemplo:
 - `feat/clase-cuenta-ahorro`
 - `feat/menu-depositos`
 - Una rama `feat/<descripcion-del-cambio>` por cada aporte o funcionalidad.
- **Flujo de integración:** los Pull Requests (PR) van desde `feat/...` hacia cada rama protegida, es decir: `feat` → `dev`, `feat` → `qa` y `feat` → `prod`, según corresponda en cada caso.
- **Requisitos de cada PR:** todo Pull Request debe incluir:
 - **Descripción** del cambio o funcionalidad.
 - **Asignación** (assignee).
 - **Revisores** (reviewers).
 - **Labels** (etiquetas).
- **Evaluación de conversaciones:** se calificarán las conversaciones y revisiones que se realicen en la rama `dev` (comentarios en PR, respuestas, mejoras sugeridas).
- Commits con mensajes claros y en español o inglés (ej.: “*Añade clase CuentaAhorro con interés y meta*”).

4. Criterios de evaluación y ponderación

La calificación se distribuye de la siguiente forma:

Criterio	Ponderación
Programación Orientada a Objetos (POO)	20 %
Uso de Git (ramas protegidas, feat , commits, flujo)	40 %
Estilo PEP 8 y calidad de código	20 %
Entrega, documentación y funcionamiento del programa	20 %
Total	100 %

4.1 Detalle por criterio

- **POO (20 %):** Diseño de clases, herencia, encapsulamiento, type hints y organización en módulos.
- **Git (40 %):** Repositorio con nombre `backend-<proyecto>`; ramas `dev`, `qa`, `prod`; flujo `feat` → `dev/qa/prod`; PR con descripción, asignación, revisores y labels; conversaciones y revisiones en `dev`; commits coherentes y mensajes descriptivos.
- **PEP 8 y calidad (20 %):** Uso obligatorio de Black, cumplimiento de convenciones de estilo, nombres claros, docstrings y código legible.
- **Entrega y funcionamiento (20 %):** URL del repositorio y nombre de cuenta GitHub registrados en el Excel del equipo de Teams antes del cierre; repositorio accesible; **README** o instrucciones básicas; programa que ejecute según lo pedido.

5. Referencia de ejemplo

Se puede tomar como referencia el proyecto de la carpeta `02-Ejemplo-examen-1`: clases `Cuenta`, `CuentaAhorro` y `CuentaCorriente`, menú en `main.py`, validaciones sin `try/except` y estructura `src/entities/`. El enunciado concreto del problema a implementar (dominio del negocio) puede ser indicado por el docente de forma adicional.

Documento generado para el curso de Programación de Software. Fechas y porcentajes aplicables al periodo indicado.